

LAB 10

GET /products (ดู products ทั้งหมด)

GET http://localhost:8080/products

Params Authorization Headers (6) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

1

Body Cookies Headers (2) Test Results

Status: 200 OK Time: 7 ms Size: 564 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673380434-1 SEC 4)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 60,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  },
22  {
23    "id": "3",
24    "name": "Samsung Galaxy S24",
25    "category": "Electronics",
26    "brand": "Samsung",
27    "stock": 30,
28    "price": 29900.0,
29    "discountType": "SEASONAL",
30    "discountedPrice": 23920.0
31  }
32 }
```

GET /products/1 (ดู products ตาม id)

GET http://localhost:8080/products/1

Params Authorization Headers (6) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

1

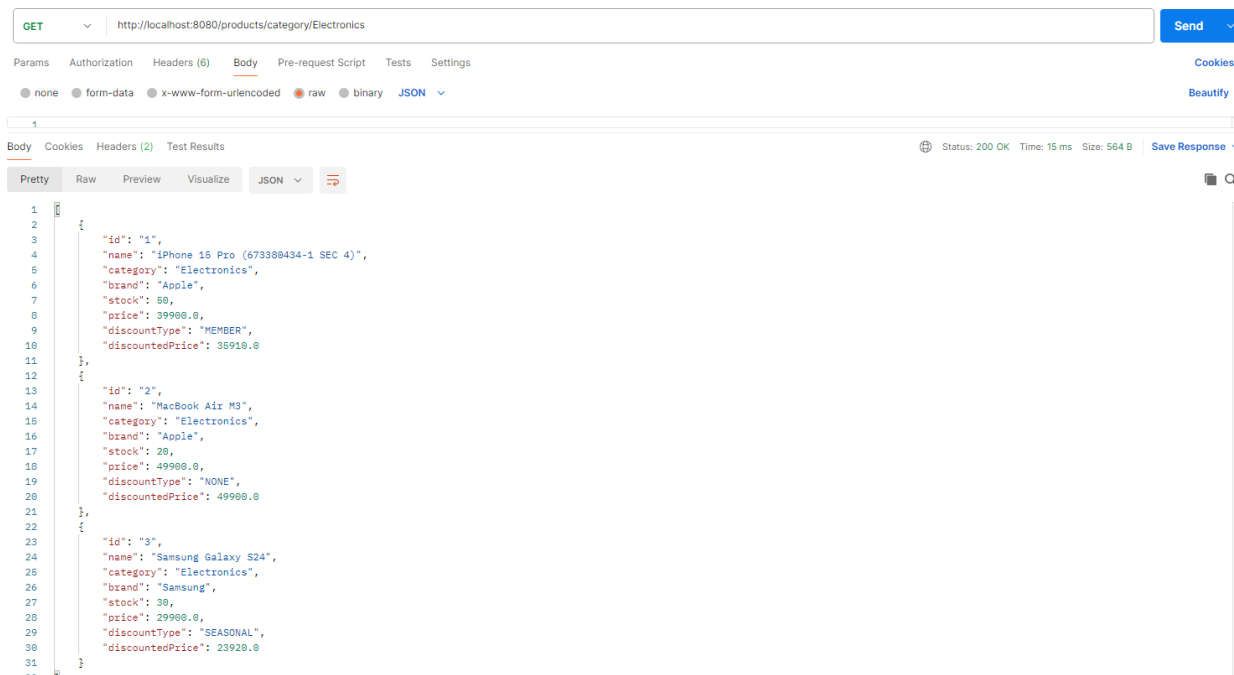
Body Cookies Headers (2) Test Results

Status: 200 OK Time: 9 ms Size: 243 B Save Response

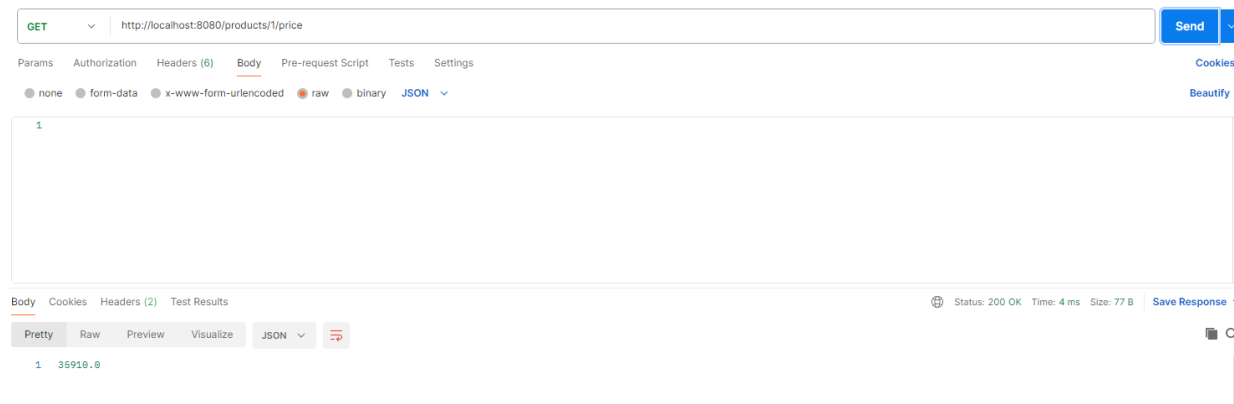
Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673380434-1 SEC 4)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 60,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

GET /products/category/Electics (ดู product ตามหมวดสินค้า)



GET /products/1/price (ดูราคาสินค้าหลังจากใช้ส่วนลด ดู product ของแต่ละ id)



POST /products (เพิ่ม products)

POST http://localhost:8080/products Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```

1 {
2   "name": "vivo x300",
3   "category": "Electronics",
4   "brand": "Vivo",
5   "stock": 10,
6   "price": 1500,
7   "discountType": "MEMBER"
8 }

```

Body Cookies Headers (2) Test Results Status: 200 OK Time: 5 ms Size: 251 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": "e48d6d4d-c696-427f-a288-edac5a74a13a",
3   "name": "vivo x300",
4   "category": "Electronics",
5   "brand": "Vivo",
6   "stock": 10,
7   "price": 1500.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 1350.0
10 }

```

DELETE /products/2 (ลบ products 2 ออก)

DELETE http://localhost:8080/products/2 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```

1

```

Body Cookies Headers (1) Test Results Status: 200 OK Time: 3 ms Size: 38 B Save Response

Pretty Raw Preview Visualize Text

```

1

```

GET /products (ดู products ทั้งหมดหลังลบ products 2 ออก)

GET http://localhost:8080/products Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```

1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673380434-1 SEC 4)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39999.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 38999.0
10 },
11 {
12   "id": "3",
13   "name": "Samsung Galaxy S24",
14   "category": "Electronics",
15   "brand": "Samsung",
16   "stock": 30,
17   "price": 29999.0,
18   "discountType": "SEASONAL",
19   "discountedPrice": 23999.0
20 },
21 {
22   "id": "e48d6d4d-c696-427f-a288-edac5a74a13a",
23   "name": "vivo x300",
24   "category": "Electronics",
25   "brand": "Vivo",
26   "stock": 10,
27   "price": 1500.0,
28   "discountType": "MEMBER",
29   "discountedPrice": 1350.0
30 }
31 }

```

1.Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Blocking คือการทำงานที่ต้องรอให้คำสั่งหนึ่งทำงานเสร็จก่อน จึงจะทำคำสั่งถัดไปได้

Reactive Programming คือการทำงานแบบไม่ต้องรอผลลัพธ์ ทำให้ระบบสามารถไปรับงานอื่นต่อได้ แล้วค่อยนำผลลัพธ์กลับมาจัดการเมื่อพร้อม

ดังนั้น Reactive เหมาะกับระบบที่ต้องรองรับงานหรือผู้ใช้จำนวนมาก

2.Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono ใช้สำหรับข้อมูล 0 หรือ 1 ค่า เช่น ค้นหา Product จาก ID ซึ่งปกติจะได้ Product เดียว

Flux ใช้สำหรับข้อมูล 0 ถึงหลายค่า เช่น การดึง Product ทั้งหมด

เลือกใช้ตามจำนวนข้อมูลที่ต้องการรับกลับมา

3.WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ เป็นเครื่องมือสำหรับเรียก REST API แบบ Reactive โดย Method Chain ทำงานตามลำดับดังนี้

.get() กำหนดว่าจะส่งคำขอแบบ GET

.uri() กำหนด URL หรือ Endpoint ที่ต้องการเรียก

.retrieve() ส่ง Request และเตรียมรับ Response

.bodyToMono() แปลง Response เป็นข้อมูลแบบ Mono เมื่อคาดว่าจะได้ข้อมูลเดียว

.bodyToFlux() แปลง Response เป็นข้อมูลแบบ Flux เมื่อคาดว่าจะได้หลายข้อมูล

4.Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ class ProductController.java

```

45 @GetMapping("/{id}")
46 public Mono<Product> getId(@PathVariable String id) {
47     return service.getId(id);
48 }
49
50 // =====
51 // X TODO: เพิ่ม method body ด้านล่างนี้
52 // =====
53
54 /**
55  * GET /products
56  * TODO: คืน Flux<Product> ที่รายการ
57  *
58  * Hint: เรียก service.getAll()
59  * พดสอบ: GET http://localhost:8080/products
60  */
61 @GetMapping
62 public Flux<Product> getAll() {
63     // TODO: เพิ่ม code ตรงนี้
64     return service.getAll(); // + แกนรรพดี
65 }
66
67 /**
68  * POST /products
69  * TODO: รับ Product จาก request body แล้วบันทึก
70  *
71  * Hint: เรียก service.save(product)
72  * พดสอบ: POST http://localhost:8080/products
73  * Body: { "name": "...", "price": 999.0, ... }
74  */
75 @PostMapping
76 public Mono<Product> save(@RequestBody Product product) {
77     // TODO: เพิ่ม code ตรงนี้
78     return service.save(product); // + แกนรรพดี
79 }
80
81 /**
82  * DELETE /products/{id}
83  * TODO: ลบ Product และคืน MonoVoid
84  *
85  * Hint: เรียก service.delete(id)
86  * พดสอบ: DELETE http://localhost:8080/products/1
87  */
88 @DeleteMapping("/{id}")
89 public MonoVoid delete(@PathVariable String id) {
90     // TODO: เพิ่ม code ตรงนี้
91     return service.delete(id); // + แกนรรพดี
92 }
93
94 /**
95  * GET /products/category/{category}
96  * TODO: คืน Flux<Product> ที่กรองตาม category
97  *
98  * Hint: เรียก service.getByCategory(category)
99  * พดสอบ: GET http://localhost:8080/products/category/Electronics
100  */
101 @GetMapping("/{category}/{category}")
102 public Flux<Product> getByCategory(@PathVariable String category) {
103     // TODO: เพิ่ม code ตรงนี้
104     return service.getByCategory(category); // + แกนรรพดี
105 }
106
107 /**
108  * GET /products/{id}/price
109  * TODO: คืน Mono<Double> ราคาหลังส่วนลด
110  *
111  * Hint: เรียก service.getDiscountedPrice(id)
112  * พดสอบ: GET http://localhost:8080/products/1/price
113  */
114 @GetMapping("/{id}/price")
115 public Mono<Double> getDiscountedPrice(@PathVariable String id) {
116     // TODO: เพิ่ม code ตรงนี้
117     return service.getDiscountedPrice(id); // + แกนรรพดี
118 }
119 }

```

Controller เป็นจุดรับ HTTP Request เช่น GET, POST, DELETE แล้วส่งต่อให้ Service ประมวลผล เมื่อได้ผลลัพธ์ก็ส่งกลับให้ Client ในรูปแบบ Mono หรือ Flux

Class ProductService.java

```

26  */
27  @Service
28  public class ProductService {
29
30      // -- Constructor Injection (DIP - SOLID) -----
31      private final ProductRepository repository;
32
33      public ProductService(ProductRepository repository) {
34          this.repository = repository;
35      }
36
37      // -- 1. ค้นหา Product 1 รายการ -----
38      /**
39       * TODO: เรียก repository.findById(id) แล้วคืนผล
40       * ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
41       *
42       * Hint: repository.findById(id)
43       *       .switchIfEmpty(Mono.error(new RuntimeException(...)))
44       */
45      public Mono<Product> getById(String id) {
46          return repository.findById(id)
47              .switchIfEmpty(
48                  Mono.error(
49                      new RuntimeException("Product not found: " + id)
50                  )
51              );
52      }
53
54      // -- 2. ค้นหา Product ทั้งหมด -----
55      /**
56       * TODO: เรียก repository.findAll() แล้วคืนผล
57       */
58      public Flux<Product> getAll() {
59          return repository.findAll();
60      }
61
62      // -- 3. บันทึก Product -----
63      /**
64       * TODO: เรียก repository.save(product) แล้วคืนผล
65       *
66       * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่
67       * Hint: java.util.UUID.randomUUID().toString()
68       */
69      public Mono<Product> save(Product product) {
70
71          if (product.getId() == null) {
72              product.setId(UUID.randomUUID().toString());
73          }
74
75          return repository.save(product);
76      }
77
78      // -- 4. ลบ Product -----
79      /**
80       * TODO: เรียก repository.deleteById(id) แล้วคืนผล
81       */
82      public Mono<Void> delete(String id) {
83          return repository.deleteById(id);
84      }
85
86      // -- 5. ค้นหาตาม category -----
87      /**
88       * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
89       */
90      public Flux<Product> getByCategory(String category) {
91          return repository.findByCategory(category);
92      }
93
94      // -- 6. ค้นหาเฉพาะสินค้าลด -----
95      /**
96       * TODO: หา Product จาก id แล้วคืน discountedPrice
97       *
98       * Hint: getById(id)
99       *       .map(p -> p.getDiscountedPrice())
100      */
101      public Mono<Double> getDiscountedPrice(String id) {
102          return getById(id)
103              .map(p -> p.getDiscountedPrice());
104      }
105  }

```

Service รับคำขอจาก Controller แล้วนำไปประมวลผล เช่น ตรวจสอบข้อมูล สร้าง ID ใหม่ หรือจัดการกรณีที่ไม่พบ Product จากนั้นจึงเรียก Repository เพื่อจัดการข้อมูล

Class ProductRepository.java

```

24 public class ProductRepository {
25
26     // -- In-memory storage -----
27     private final Map<String, Product> store = new ConcurrentHashMap<>();
28
29     // -- Constructor: ใส่ข้อมูลสินค้า -----
30     public ProductRepository() {
31         store.put(key: "1", new Product(id: "1", name: "iPhone 15 Pro (673380434-1 SEC 4)",
32             category: "Electronics", brand: "Apple", stock: 50, price: 39900.0, discountType: "MEMBER"));
33         store.put(key: "2", new Product(id: "2", name: "MacBook Air M3",
34             category: "Electronics", brand: "Apple", stock: 20, price: 49900.0, discountType: "NONE"));
35         store.put(key: "3", new Product(id: "3", name: "Samsung Galaxy S24",
36             category: "Electronics", brand: "Samsung", stock: 30, price: 29900.0, discountType: "SEASONAL"));
37     }
38
39     // -- 1. หา Product 1 รายการ -----
40     /**
41      * TODO: ค้น Mono<Product> จาก store โดยใช้ id
42      * ถ้าไม่พบให้คืน Mono.empty()
43      *
44      * Hint: store.get(id) คืน Product หรือ null
45      * ถ้า null ให้คืน Mono.empty()
46      * ถ้ามีค่าให้คืน Mono.just(product)
47      */
48     public Mono<Product> findById(String id) {
49         Product product = store.get(id);
50
51         if (product == null) {
52             return Mono.empty();
53         }
54
55         return Mono.just(product);
56     }
57
58     // -- 2. หา Product ทั้งหมด -----
59     /**
60      * TODO: ค้น Flux<Product> ของทุกตัวภายใน store
61      *
62      * Hint: store.values() คืน Collection<Product>
63      * ใช้ Flux.fromIterable(...) แปลงเป็น Flux
64      */
65     public Flux<Product> findAll() {
66         return Flux.fromIterable(store.values());
67     }
68
69     // -- 3. บันทึก Product -----
70     /**
71      * TODO: บันทึก product ลง store แล้วคืน Mono<Product>
72      *
73      * Hint: store.put(product.getId(), product)
74      * แล้วคืน Mono.just(product) คืนค่า
75      */
76     public Mono<Product> save(Product product) {
77         store.put(product.getId(), product);
78         return Mono.just(product);
79     }
80
81     // -- 4. ลบ Product -----
82     /**
83      * TODO: ลบ product จาก store แล้วคืน Mono<Void>
84      *
85      * Hint: store.remove(id)
86      * แล้วคืน Mono.empty() คืนค่า (Mono<Void>)
87      */
88     public Mono<Void> deleteById(String id) {
89         store.remove(id);
90         return Mono.empty();
91     }
92
93     // -- 5. ค้นหาตาม category -----
94     /**
95      * TODO: ค้น Flux<Product> ที่ category ตรงกัน
96      *
97      * Hint: findAll()
98      * .filter(p -> p.getCategory().equalsIgnoreCase(category))
99      */
100     public Flux<Product> findByCategory(String category) {
101         return findAll()
102             .filter(p -> p.getCategory().equalsIgnoreCase(category));
103     }
104 }

```

Repository เป็นชั้นที่ติดต่อกับแหล่งเก็บข้อมูลโดยตรง เช่น ค้นหา เพิ่ม ลบ และกรอง Product โดยใช้ Mono เมื่อได้ข้อมูลเดียว และ Flux เมื่อได้หลายข้อมูล